Career Navigator Platform

A Comprehensive Full-Stack Mentorship Application

Technical Documentation and Architectural Overview

Development Team
Tuheu Tchoubi Pempeme Moussa Fahdil (Lead Backend Developer)
Yuyar Lea-Babara (Lead Flutter Developer)
NDZEKA GETRUDE BERINYUY (UI/UX Designer)
Instructor: [Teacher's Name]

Course: Software Engineering Project
Date: May 7, 2026

Contents

1 Project Overview **4**
1.1 What is Career Navigator? 4
1.2 Core Features 4
1.3 Technology Stack 5

2 System Architecture **5**
2.1 Overall Architecture 5
2.2 High-Level System Diagram 5

3 Design Patterns Implemented **7**
3.1 Overview of Design Patterns Used 7
3.2 Detailed Pattern Explanations 7
3.2.1 Singleton Pattern 7
3.2.2 Factory Pattern 8
3.2.3 Repository Pattern 8
3.2.4 Service Layer Pattern 8
3.2.5 Builder Pattern 9
3.2.6 Observer Pattern (WebSocket) 9

4 Software Engineering Principles Applied **10**
4.1 SOLID Principles 10
4.1.1 S - Single Responsibility Principle 10
4.1.2 O - Open/Closed Principle 10
4.1.3 L - Liskov Substitution Principle 10
4.1.4 I - Interface Segregation Principle 11
4.1.5 D - Dependency Inversion Principle 11
4.2 Other Software Engineering Principles 11
4.2.1 DRY (Don't Repeat Yourself) 11
4.2.2 KISS (Keep It Simple, Stupid) 12
4.2.3 YAGNI (You Aren't Gonna Need It) 12
4.2.4 Separation of Concerns 12
4.2.5 Convention over Configuration 12

5 Project Structure and File Organization **12**
5.1 Frontend Structure (Flutter) 12
5.2 Backend Structure (Flask) 14

6 Database Schema **15**
6.1 Entity Relationship Diagram Overview 15
6.2 Tables Summary 16
6.3 Key Relationships 16

7 API Endpoints Documentation **17**
7.1 Authentication Endpoints 17
7.2 Profile Endpoints 17
7.3 Job Endpoints 18

7.4 Chat Endpoints 18

8 Token Refresh Mechanism **18**
8.1 How JWT Token Refresh Works 18
8.2 Frontend Implementation 19

9 Real-Time Chat Architecture **19**
9.1 WebSocket vs REST Fallback 19
9.2 Typing Indicators Implementation 20

10 Video Calling with Agora SDK **21**
10.1 Video Call Flow 21

11 State Management with Provider **21**
11.1 Provider Architecture 21

12 Security Measures Implemented **22**
12.1 Security Features 22
12.2 Password Hashing Example 23
12.3 Parameterized Query Example 23

13 Deployment Configuration **23**
13.1 PM2 Ecosystem Configuration 23
13.2 Nginx Configuration for WebSocket 24

14 What We Have Accomplished **24**
14.1 Completed Features 24
14.2 Technical Achievements 25

15 Lessons Learned **26**
15.1 Technical Lessons 26
15.2 Challenges Overcome 26

16 Future Improvements **26**
16.1 Planned Enhancements 26

17 Conclusion **27**

1 Project Overview

1.1 What is Career Navigator?

Career Navigator is a comprehensive career mentorship platform that connects ambitious **job seekers** with experienced **mentors**. The platform enables personalized career guidance, skill development, professional networking, job searching, and real-time communication — all in one place.

1.2 Core Features

- **User Authentication**: Registration, email verification (OTP), login, JWT-based authentication with token refresh
- **Dual User Roles**: Job Seekers and Mentors with role-specific dashboards
- **Profile Management**: Complete CRUD operations for personal information, education history, and work experience
- **Job System**: Browse jobs with filters, search, and apply with cover letters
- **Mentorship Requests**: Job seekers can send mentorship requests to mentors
- **Real-time Chat**: WebSocket-based instant messaging with typing indicators and online status
- **Video Calling**: One-on-one video sessions using Agora SDK
- **Notifications**: In-app notifications for all user interactions
- **Search**: Find mentors and job seekers by name, skills, or expertise
- **Feedback System**: Users can submit feedback with ratings
- **Admin Panel**: User management and feedback review (coming soon)
- **Theme Support**: Dark and light mode with persistent user preference

1.3 Technology Stack

Component	Technology	Version
Frontend Framework	Flutter	3.16+
Backend Framework	Flask (Python)	3.1.3
Database	MySQL	8.0+
Real-time Communication	Socket.IO	4.9+
Video Calling	Agora RTC SDK	6.3.2
Authentication	JWT (JSON Web Tokens)	4.7.1
Email Service	Brevo (Sendinblue) API	7.6.0
File Storage	Cloudinary / Local	-
State Management	Provider	6.1.2
HTTP Client	Dio / HTTP	1.2.2
Process Management	PM2	Latest

Table 1: Technology Stack

2 System Architecture

2.1 Overall Architecture

The Career Navigator application follows a **three-tier architecture** with clear separation between presentation, business logic, and data layers.

Presentation Layer (Flutter Frontend) *UI Screens, Widgets, State Management*
↓ HTTP/WebSocket
Business Logic Layer (Flask Backend) *Controllers, Services, Repositories*
↓ SQL Queries
Data Layer (MySQL Database) *Tables, Relations, Indexes*

Figure 1: Three-Tier Architecture

2.2 High-Level System Diagram

```
+-------------------------------------------------------------+
|                    CLIENT (Flutter App)                      |
|  +-------+ +-------+ +-------+ +-------+ +-------+          |
|  | Auth  | |Profile| | Jobs  | | Chat  | | Video |          |
|  | Screens| |Screens| |Screens| |Screens| |Screens|         |
|  +-------+ +-------+ +-------+ +-------+ +-------+          |
|                                                              |
|  +-----------------------------------------------------+    |
|  |            Provider State Management                |    |
|  +-----------------------------------------------------+    |
+-------------------------------------------------------------+
```

```
                              |
                              | HTTP/HTTPS + WebSocket
                              v
+-------------------------------------------------------------+
|                   BACKEND (Flask API)                       |
|  +---------------------------------------------------------+  |
|  |                 Middleware Layer                        |  |
|  |  +----------+ +----------+ +----------+ +----------+  |  |
|  |  |  CORS    | |  JWT     | |  Rate    | | Request  |  |  |
|  |  |          | |  Auth    | | Limiter  | |  Logger  |  |  |
|  |  +----------+ +----------+ +----------+ +----------+  |  |
|  +---------------------------------------------------------+  |
|                              |                               |
|  +---------------------------------------------------------+  |
|  |                 Controller Layer                        |  |
|  |  +------+ +------+ +------+ +------+ +------+         |  |
|  |  | Auth | |Profile| | Job  | | Chat | | Video|         |  |
|  |  |Controller|Controller|Controller|Controller|Controller| |
|  |  +------+ +------+ +------+ +------+ +------+         |  |
|  +---------------------------------------------------------+  |
|                              |                               |
|  +---------------------------------------------------------+  |
|  |                  Service Layer                          |  |
|  |  +--------+ +--------+ +--------+ +--------+         |  |
|  |  | Auth   | | Email  | | Token  | | Chat   |         |  |
|  |  | Service| | Service| | Service| | Service|         |  |
|  |  +--------+ +--------+ +--------+ +--------+         |  |
|  +---------------------------------------------------------+  |
|                              |                               |
|  +---------------------------------------------------------+  |
|  |                Repository Layer                         |  |
|  |  +------+ +------+ +------+ +------+ +------+         |  |
|  |  | User | | Job  | | Chat | | Notif| | Video|         |  |
|  |  | Repo | | Repo | | Repo | | Repo | | Repo |         |  |
|  |  +------+ +------+ +------+ +------+ +------+         |  |
|  +---------------------------------------------------------+  |
+-------------------------------------------------------------+
                              |
                              | SQL
                              v
+-------------------------------------------------------------+
|                   DATABASE (MySQL)                          |
|  +---------------------------------------------------------+  |
|  |  Tables: users, job_seekers, mentor_profiles,           |  |
|  |  education, work_experience, job_listings,              |  |
|  |  conversations, messages, notifications, etc.           |  |
|  +---------------------------------------------------------+  |
+-------------------------------------------------------------+
```

3 Design Patterns Implemented

3.1 Overview of Design Patterns Used

The following table summarizes the design patterns implemented across the project:

Pattern	Where Used	Purpose
Singleton	Database Connection Pool, Settings Manager, Logger Factory	Ensure single instance throughout application
Factory	ResponseFactory, LoggerFactory	Create objects without specifying concrete classes
Repository	All data access classes (UserRepository, JobRepository, etc.)	Abstract data access logic from business logic
Service Layer	AuthService, EmailService, TokenService	Encapsulate business logic
Dependency Injection	Controllers receive services via constructor	Loose coupling and testability
Strategy	Token validation strategies (Access vs Refresh)	Define family of algorithms
Observer	WebSocket event handlers (Socket.IO)	One-to-many dependency notification
Template Method	BaseModel, BaseRepository abstract classes	Define skeleton of algorithm
Builder	QueryBuilder class	Construct complex SQL queries step by step
Decorator	Middleware (auth, rate limiting, logging)	Add responsibilities dynamically
MVC (Model-View-Controller)	Models, Controllers, JSON responses	Separate concerns
DTO (Data Transfer Object)	ResponseFactory success/error methods	Transfer data between layers

Table 2: Design Patterns Summary

3.2 Detailed Pattern Explanations

3.2.1 Singleton Pattern

The Singleton pattern ensures a class has only one instance and provides a global point of access to it.

```
class DatabaseConnectionPool:
    _instance = None
    _lock = Lock()

    def __new__(cls):
        if cls._instance is None:
            with cls._lock:
                if cls._instance is None:
```

```
                    cls._instance = super().__new__(cls)
                    cls._instance._initialize()
        return cls._instance
```

Listing 1: Singleton Pattern in Database Connection Pool

Why used: Database connections are expensive resources. Having a single connection pool across the entire application prevents resource exhaustion and ensures efficient connection management.

3.2.2 Factory Pattern

The Factory pattern provides an interface for creating objects without specifying their concrete classes.

```
class ResponseFactory:
    @staticmethod
    def success(data=None, message="OK"):
        return {"success": True, "message": message, "data": data}

    @staticmethod
    def error(message="Error", status_code=400):
        return {"success": False, "message": message, "data": None}
```

Listing 2: Factory Pattern in Response Creation

Why used: Ensures consistent API response format across all endpoints, making frontend parsing predictable and error handling uniform.

3.2.3 Repository Pattern

The Repository pattern mediates between the domain and data mapping layers, acting like an in-memory domain object collection.

```
class BaseRepository(ABC):
    def find_by_id(self, id: int) -> Optional[Dict]:
        pass

    def create(self, data: Dict) -> int:
        pass

    def update(self, id: int, data: Dict) -> bool:
        pass

class UserRepository(BaseRepository):
    def find_by_email(self, email: str) -> Optional[Dict]:
        return self._query("SELECT * FROM users WHERE email = %s", (
    email,))
```

Listing 3: Repository Pattern

Why used: Centralizes data access logic, makes testing easier (can mock repositories), and allows changing database implementation without affecting business logic.

3.2.4 Service Layer Pattern

The Service Layer pattern defines an application boundary with a set of available operations.

```python
class AuthService:
    def __init__(self):
        self.user_repo = UserRepository()
        self.email_service = EmailService()
        self.token_service = TokenService()

    def register(self, email, password):
        # Business logic here
        if not self.validators.validate_email(email):
            return ResponseFactory.error("Valid email required")
        # ... more logic
```

Listing 4: Service Layer Pattern

Why used: Encapsulates business logic separately from controllers and repositories, promotes code reuse, and makes the system more maintainable.

3.2.5 Builder Pattern

The Builder pattern constructs complex objects step by step.

```python
class QueryBuilder:
    def __init__(self, table: str):
        self._table = table
        self._where_conditions = []
        self._order_by = []

    def where(self, condition: str, *params):
        self._where_conditions.append(condition)
        return self

    def order_by(self, field: str, direction: str = "ASC"):
        self._order_by.append(f"{field} {direction}")
        return self

    def build_select(self) -> tuple:
        query = f"SELECT * FROM {self._table}"
        if self._where_conditions:
            query += f" WHERE {' AND '.join(self._where_conditions)}"
        return query
```

Listing 5: Builder Pattern for SQL Queries

Why used: Prevents SQL injection by safely building parameterized queries, makes complex query construction readable and maintainable.

3.2.6 Observer Pattern (WebSocket)

The Observer pattern defines a one-to-many dependency where when one object changes state, all its dependents are notified.

```python
@socketio.on('send_message')
def handle_send_message(data):
    # Save message to database
    # Notify recipient (observer)
    if recipient_online:
        emit('new_message', message_data, room=f"user_{recipient_id}")
```

Listing 6: Observer Pattern in WebSocket

Why used: Enables real-time communication where message senders don't need to know about recipients explicitly.

4 Software Engineering Principles Applied

4.1 SOLID Principles

4.1.1 S - Single Responsibility Principle

Each class has exactly one reason to change.

- **AuthController** only handles HTTP requests for authentication
- **AuthService** only contains authentication business logic
- **UserRepository** only handles database operations for users
- **EmailService** only manages email sending

```python
# Good - Each class has one responsibility
class EmailService:      # Only sends emails
    def send_email(self): pass

class AuthService:       # Only auth logic
    def register(self): pass

class UserRepository:    # Only user DB operations
    def find_by_email(self): pass
```

Listing 7: Single Responsibility Example

4.1.2 O - Open/Closed Principle

Classes are open for extension but closed for modification.

```python
# Base class - closed for modification
class BaseRepository(ABC):
    @abstractmethod
    def create(self, data): pass

# Extended - open for extension
class UserRepository(BaseRepository):
    def create(self, data):
        # Specific implementation
        pass
```

Listing 8: Open/Closed Example

4.1.3 L - Liskov Substitution Principle

Derived classes must be substitutable for their base classes.

```python
# Any repository can be used where BaseRepository is expected
def process_repository(repo: BaseRepository):
    return repo.find_by_id(1)

# These all work because they follow the same contract
user_repo = UserRepository()     # + Substitutable
job_repo = JobRepository()       # + Substitutable
chat_repo = ChatRepository()     # + Substitutable
```

Listing 9: Liskov Substitution Example

4.1.4 I - Interface Segregation Principle

Clients should not be forced to depend on interfaces they do not use.

```python
# Separate small interfaces instead of one large interface
class Readable(ABC):
    @abstractmethod
    def find_by_id(self, id): pass

class Writeable(ABC):
    @abstractmethod
    def create(self, data): pass
    @abstractmethod
    def update(self, id, data): pass

# Repository implements only what it needs
class UserRepository(Readable, Writeable):
    pass
```

Listing 10: Interface Segregation Example

4.1.5 D - Dependency Inversion Principle

Depend on abstractions, not concretions.

```python
# AuthService depends on abstraction (BaseRepository), not concrete class
class AuthService:
    def __init__(self, user_repo: BaseRepository):  # Abstraction
        self.user_repo = user_repo                  # Not concrete

# Dependency injection
auth_service = AuthService(UserRepository())  # Works with any repository
```

Listing 11: Dependency Inversion Example

4.2 Other Software Engineering Principles

4.2.1 DRY (Don't Repeat Yourself)

- BaseRepository provides common database operations (find_by_id, find_all, delete)
- BaseModel provides common serialization methods (to_dict, from_dict)
- ResponseFactory ensures consistent API response format

4.2.2 KISS (Keep It Simple, Stupid)

- Simple, readable function names
- Clear separation of concerns
- No unnecessary abstractions
- Straightforward error handling

4.2.3 YAGNI (You Aren't Gonna Need It)

- Only implemented features requested in requirements
- No speculative "future-proofing" code
- Features added only when needed

4.2.4 Separation of Concerns

```
+---------------------------------------------------+
|  Controllers (HTTP request/response)              |
+---------------------------------------------------+
|  Services (Business logic)                        |
+---------------------------------------------------+
|  Repositories (Data access)                       |
+---------------------------------------------------+
|  Models (Data representation)                     |
+---------------------------------------------------+
```

4.2.5 Convention over Configuration

- Table names follow plural convention (users, jobs, messages)
- Foreign keys follow table_id convention (user_id, job_id)
- Timestamp columns: created_at, updated_at
- API endpoints follow REST conventions (/resource/id)

5 Project Structure and File Organization

5.1 Frontend Structure (Flutter)

```
lib/
+-- main.dart                          # Application entry point
+-- core/
|   +-- constants/                     # App constants and endpoints
|   |   +-- app_constants.dart
|   +-- themes/                        # Dark/Light theme configuration
|   |   +-- app_theme.dart
|   +-- utils/                         # Helper functions
```

```
|   |   +-- helpers.dart
|   |   +-- validators.dart
|   +-- services/                     # API and Socket services
|       +-- api_service.dart          # HTTP requests with token refresh
|       +-- socket_service.dart       # WebSocket real-time chat
|
+-- data/
|   +-- datasources/
|   |   +-- local/                    # Local storage
|   |   |   +-- token_store.dart      # JWT token persistence
|   |   +-- remote/                   # Remote data sources
|   |       +-- api_service.dart
|   +-- models/                       # Data models (6 files)
|   |   +-- user_model.dart
|   |   +-- job_model.dart
|   |   +-- chat_model.dart
|   |   +-- notification_model.dart
|   |   +-- feedback_model.dart
|   |   +-- video_session_model.dart
|   +-- repositories/                 # Repository pattern (8 files)
|       +-- auth_repository.dart
|       +-- user_repository.dart
|       +-- job_repository.dart
|       +-- chat_repository.dart
|       +-- mentor_repository.dart
|       +-- notification_repository.dart
|       +-- feedback_repository.dart
|       +-- video_repository.dart
|
+-- providers/                        # State management (8 files)
|   +-- auth_provider.dart
|   +-- theme_provider.dart
|   +-- user_provider.dart
|   +-- job_provider.dart
|   +-- chat_provider.dart
|   +-- notification_provider.dart
|   +-- video_provider.dart
|   +-- search_provider.dart
|
+-- screens/                          # UI Screens (46 files)
|   +-- auth/                         # 5 auth screens
|   +-- dashboard/                    # 3 dashboards
|   +-- profile/                      # 5 profile screens
|   +-- jobs/                         # 4 job screens
|   +-- chat/                         # 3 chat screens
|   +-- video/                        # 3 video screens
|   +-- mentors/                      # 4 mentor screens
|   +-- search/                       # 1 search screen
```

```
|   +-- notifications/                # 1 notification screen
|   +-- feedback/                     # 1 feedback screen
|   +-- settings/                     # 4 settings screens
|   +-- admin/                        # 4 admin screens
|
+-- widgets/                          # Reusable components (21 files)
|   +-- shared/                       # 9 shared widgets
|   +-- chat/                         # 3 chat widgets
|   +-- jobs/                         # 3 job widgets
|   +-- profile/                      # 3 profile widgets
|   +-- mentors/                      # 3 mentor widgets
|
+-- utils/                            # Utility functions
    +-- date_formatter.dart
    +-- image_picker_helper.dart
```

5.2 Backend Structure (Flask)

```
career_navigator_backend/
+-- app.py                            # Main application entry point
+-- requirements.txt                  # Python dependencies
+-- .env                              # Environment variables
+-- ecosystem.config.js               # PM2 deployment config
|
+-- config/                           # Configuration (4 files)
|   +-- settings.py                   # Centralized settings (Singleton)
|   +-- database.py                   # Connection pool + QueryBuilder
|   +-- logging_config.py             # Logger factory
|
+-- models/                           # Data models (7 files)
|   +-- base_model.py                 # Abstract base model
|   +-- user_model.py                 # User, JobSeeker, Mentor
|   +-- job_model.py                  # JobListing, JobApplication
|   +-- chat_model.py                 # Conversation, Message
|   +-- notification_model.py         # Notification model
|   +-- feedback_model.py             # Feedback model
|   +-- video_session_model.py        # Video session model
|
+-- repositories/                     # Data access (8 files)
|   +-- base_repository.py            # Abstract repository
|   +-- user_repository.py            # User CRUD operations
|   +-- job_repository.py             # Job CRUD operations
|   +-- chat_repository.py            # Chat operations
|   +-- notification_repository.py    # Notification operations
|   +-- feedback_repository.py        # Feedback operations
|   +-- video_repository.py           # Video session operations
|   +-- token_repository.py           # Token storage
|
```

```
+-- services/                          # Business logic (6 files)
|   +-- auth_service.py                # Authentication logic
|   +-- email_service.py               # Brevo email integration
|   +-- token_service.py               # JWT token management
|   +-- validation_service.py          # Input validation
|   +-- chat_service.py                # Chat business logic
|   +-- file_service.py                # File upload (Cloudinary)
|
+-- controllers/                       # HTTP handlers (10 files)
|   +-- auth_controller.py             # Auth endpoints
|   +-- profile_controller.py          # Profile endpoints
|   +-- job_controller.py              # Job endpoints
|   +-- chat_controller.py             # Chat REST fallback
|   +-- mentor_controller.py           # Mentor endpoints
|   +-- search_controller.py           # Search endpoints
|   +-- video_controller.py            # Video endpoints
|   +-- admin_controller.py            # Admin endpoints
|   +-- feedback_controller.py         # Feedback endpoints
|   +-- notification_controller.py     # Notification endpoints
|
+-- middleware/                        # Request interceptors (5 files)
|   +-- auth_middleware.py             # JWT + role-based auth
|   +-- error_handler.py               # Global error handlers
|   +-- request_logger.py              # Request/response logging
|   +-- rate_limiter.py                # Rate limiting
|   +-- cors_middleware.py             # CORS configuration
|
+-- sockets/                           # WebSocket handlers (3 files)
|   +-- chat_socket.py                 # Real-time chat events
|   +-- events.py                      # User online tracking
|
+-- utils/                             # Utilities (4 files)
|   +-- response_factory.py            # API response factory
|   +-- validators.py                  # Input validation
|   +-- helpers.py                     # Helper functions
|   +-- decorators.py                  # Custom decorators
|
+-- migrations/
    +-- schema.sql                     # Complete database schema
```

6 Database Schema

6.1 Entity Relationship Diagram Overview

```
+-----------------------------------------------------+
|                        users                        |
|  id, email, password_hash, full_name, role, ...     |
+-----------------+-----------+-----------+--------+
```

```
                                |           |           |
        +-----------------------+-----------+-----------+-------------------+
        |                       |           |           |                   |
        v                       v           v           v                   v
+---------------+       +-----------+ +-----------+ +-----------+     +----------
| job_seekers   |       |mentor_profiles| | education | |   work_   |     |  ment
|               |       |             | |           | |experience |     |  reques
+---------------+       +-----------+ +-----------+ +-----------+     +----------
        |                       |                                           |
        |                       |                                           |
        v                       v                                           v
+---------------+       +-----------+                                 +----------
|job_applications|       |conversations|<--------------------------->|  messages
+-------+-------+       +------+-----+                               +-----------
        |                       |
        v                       v
+---------------+       +-----------+
| job_listings  |       |notifications|
+---------------+       +-----------+
```

6.2 Tables Summary

Table Name	Row Count Est	Purpose
users	10K+	Stores user accounts, authentication, roles
job_seekers	10K+	Extended profile for job seekers
mentor_profiles	5K+	Extended profile for mentors
education	50K+	Education history for all users
work_experience	50K+	Work history for all users
job_listings	5K+	Job postings by admins/mentors
job_applications	50K+	Job applications from job seekers
mentor_requests	20K+	Mentorship requests between users
conversations	10K+	Chat threads between users
messages	500K+	Individual chat messages
notifications	100K+	User notifications
video_sessions	10K+	Video call records
feedback	5K+	User feedback submissions

Table 3: Database Tables Overview

6.3 Key Relationships

- **users** → **job_seekers**: One-to-one (job seeker profile)
- **users** → **mentor_profiles**: One-to-one (mentor profile)
- **users** → **education**: One-to-many (education history)

- **users** → **work_experience**: One-to-many (work history)
- **users** → **conversations**: Many-to-many (via user_a_id, user_b_id)
- **conversations** → **messages**: One-to-many (chat messages)
- **users** → **mentor_requests**: One-to-many (as seeker or mentor)
- **users** → **notifications**: One-to-many (notifications)
- **job_listings** → **job_applications**: One-to-many (applications)

7 API Endpoints Documentation

7.1 Authentication Endpoints

Endpoint	Method	Description
/auth/register	POST	Register new user account
/auth/verify-email	POST	Verify email with OTP code
/auth/resend-code	POST	Resend verification code
/auth/login	POST	Login with email/password
/auth/refresh	POST	Refresh access token
/auth/forgot-password	POST	Request password reset
/auth/reset-password	POST	Reset password with code
/auth/change-password	POST	Change password (authenticated)
/auth/delete-account	DELETE	Delete user account
/auth/logout	POST	Logout user

Table 4: Authentication Endpoints

7.2 Profile Endpoints

Endpoint	Method	Description
/profile/me	GET	Get full user profile
/profile/setup	PUT	Initial profile setup
/profile/picture	PUT	Update profile picture URL
/upload/picture	POST	Upload profile picture file
/profile/job-seeker	PUT	Update job seeker profile
/profile/mentor	PUT	Update mentor profile
/profile/education	POST	Add education record
/profile/education/{id}	PUT	Update education record
/profile/education/{id}	DELETE	Delete education record
/profile/work-experience	POST	Add work experience
/profile/work-experience/{id}	PUT	Update work experience

/profile/work-experience/{id}	DELETE	Delete work experience

Table 5: Profile Endpoints

7.3 Job Endpoints

Endpoint	Method	Description
/jobs	GET	List jobs with filters
/jobs/{id}	GET	Get job details
/jobs/{id}/apply	POST	Apply for a job
/jobs/applications/my	GET	Get my applications

Table 6: Job Endpoints

7.4 Chat Endpoints

Endpoint	Method	Description
/chat/conversations	GET	Get all conversations
/chat/messages/{id}	GET	Get conversation messages
/chat/messages	POST	Send message (REST fallback)
WebSocket: /socket.io/	WS	Real-time chat events

Table 7: Chat Endpoints

8 Token Refresh Mechanism

8.1 How JWT Token Refresh Works

```
+-------------------------------------------------------------+
|                    TOKEN REFRESH FLOW                        |
+-------------------------------------------------------------+
|                                                              |
|  1. User logs in -> receives access_token (24h) + refresh_token (30d)    |
|                                                              |
|  2. Frontend stores both tokens in SharedPreferences         |
|                                                              |
|  3. For each API request:                                   |
|     +-----------------------------------------------------+   |
|     |  Check if access_token is expired (decode JWT exp claim) |   |
|     |                                                     |   |
|     |  if NOT expired:                                   |   |
|     |      -> Send request with access_token              |   |
|     |                                                     |   |
|     |  if EXPIRED:                                       |   |
|     |      -> Queue pending requests                      |   |
|     |      -> Call /auth/refresh with refresh_token       |   |
```

```
|     |      -> Get new access_token                    |   |
|     |      -> Retry queued requests                   |   |
|     |      -> Continue                                |   |
|     +-------------------------------------------------+   |
|                                                           |
|  4. If refresh_token also expired -> redirect to login    |
|                                                           |
+-----------------------------------------------------------+
```

8.2 Frontend Implementation

```
Future<dynamic> _requestWithAuth(Future<http.Response> Function(String)
    requestFn) async {
  if (_isTokenExpired(_accessToken!)) {
    if (_isRefreshing) {
      // Queue request while refreshing
      final completer = Completer<dynamic>();
      _pendingRequests.add({'completer': completer, 'requestFn':
   requestFn});
      return completer.future;
    }

    _isRefreshing = true;
    final refreshResult = await _refreshToken();
    _isRefreshing = false;

    if (refreshResult['success']) {
      // Retry all queued requests
      for (var pending in _pendingRequests) {
        final result = await requestFn(_accessToken!);
        pending['completer'].complete(result);
      }
      return await requestFn(_accessToken!);
    } else {
      throw Exception('Session expired. Please login again.');
    }
  }
  return await requestFn(_accessToken!);
}
```

Listing 12: Token Refresh in ApiService

9 Real-Time Chat Architecture

9.1 WebSocket vs REST Fallback

```
+-----------------------------------------------------------+
|                 REAL-TIME CHAT ARCHITECTURE                |
+-----------------------------------------------------------+
|                                                           |
|   +--------------+      WebSocket (Primary)   +-----------+ |
|   |              | <------------------------> |           | |
```

```
|   |   Flutter    |                            |   Flask   | |
|   |   Frontend   |      REST API (Fallback)   |   Backend | |
|   |              | <------------------------> |           | |
|   +--------------+                            +-----------+ |
|          |                                          |        |
|          |                                          |        |
|          v                                          v        |
|   +--------------+                            +-----------+ |
|   | SocketService|                            | ChatSocket | |
|   | (Client)     |                            | Handler    | |
|   +--------------+                            +-----------+ |
|                                                             |
|   WebSocket Events:                                         |
|   +-------------------------------------------------------+ |
|   |  - connect      - Establish connection                | |
|   |  - authenticate - Authenticate user with JWT token    | |
|   |  - send_message - Send real-time message              | |
|   |  - new_message  - Receive real-time message           | |
|   |  - mark_read    - Mark messages as read               | |
|   |  - typing       - Send/receive typing indicators      | |
|   |  - user_online  - Notify when user comes online       | |
|   |  - user_offline - Notify when user goes offline       | |
|   +-------------------------------------------------------+ |
|                                                             |
+-------------------------------------------------------------+
```

9.2 Typing Indicators Implementation

```
void _onTyping() {
  if (!_isTyping) {
    _isTyping = true;
    _chatProvider.sendTypingIndicator(widget.recipientId, true);
  }
  _typingTimer?.cancel();
  _typingTimer = Timer(const Duration(seconds: 2), _stopTyping);
}

void _stopTyping() {
  if (_isTyping) {
    _isTyping = false;
    _chatProvider.sendTypingIndicator(widget.recipientId, false);
  }
}
```

Listing 13: Typing Indicator in Chat

10 Video Calling with Agora SDK

10.1 Video Call Flow

```
+-------------------------------------------------------------+
|                    VIDEO CALL FLOW                          |
+-------------------------------------------------------------+
|                                                             |
|   JOB SEEKER                    BACKEND                    MENTOR    |
|       |                            |                          |       |
|       |   POST /video/start-session|                          |       |
|       +--------------------------->|                          |       |
|       |                            |                          |       |
|       |   {channel_name, token,    |                          |       |
|       |    app_id}                 |                          |       |
|       |<---------------------------+                          |       |
|       |                            |                          |       |
|       |                            |   Notification to Mentor |       |
|       |                            |------------------------->|       |
|       |                            |                          |       |
|       |                            |   POST /video/join-session|       |
|       |                            |<-------------------------|       |
|       |                            |                          |       |
|       |                            |   {channel_name, token,  |       |
|       |                            |    app_id}               |       |
|       |                            |------------------------->|       |
|       |                            |                          |       |
|       |   +-----------------------------------------------+       |
|       |   |         Both users join Agora channel             |       |
|       |   |         using the provided tokens                 |       |
|       |   +-----------------------------------------------+       |
|       |                            |                          |       |
|       |                            |   POST /video/end-session |       |
|       |                            |<-------------------------|       |
|       |                            |   or                      |       |
|       |   POST /video/end-session  |                          |       |
|       +--------------------------->|                          |       |
|       |                            |                          |       |
+-------------------------------------------------------------+
```

11 State Management with Provider

11.1 Provider Architecture

```
+-------------------------------------------------------------+
|                 PROVIDER STATE MANAGEMENT                   |
+-------------------------------------------------------------+
|                                                             |
```

```
|   MultiProvider (At app root)                                |
|   +-------------------------------------------------------+ |
|   |  ChangeNotifierProvider(create: (_) => ThemeProvider())       | |
|   |  ChangeNotifierProvider(create: (_) => AuthProvider())        | |
|   |  ChangeNotifierProvider(create: (_) => UserProvider())        | |
|   |  ChangeNotifierProvider(create: (_) => JobProvider())         | |
|   |  ChangeNotifierProvider(create: (_) => ChatProvider())        | |
|   |  ChangeNotifierProvider(create: (_) => NotificationProvider())| |
|   |  ChangeNotifierProvider(create: (_) => VideoProvider())       | |
|   +-------------------------------------------------------+ |
|                                   |                          |
|                                   v                          |
|   +-------------------------------------------------------+ |
|   |                 Consumer Pattern                       | |
|   |                                                       | |
|   |   Widget build(BuildContext context) {                 | |
|   |     final authProvider = context.watch<AuthProvider>();       | |
|   |     final user = authProvider.currentUser;            | |
|   |     return Text(user?.displayName ?? '');              | |
|   |   }                                                   | |
|   +-------------------------------------------------------+ |
|                                                              |
|   Provider Types:                                            |
|   +-------------------------------------------------------+ |
|   |  - Provider          - Provides value without listening       | |
|   |  - ChangeNotifier    - Base class for observable models       | |
|   |  - ChangeNotifierProvider - Provides ChangeNotifier to widgets| |
|   |  - Consumer          - Listens to changes and rebuilds       | |
|   |  - context.watch()   - Listens and rebuilds on change        | |
|   |  - context.read()    - Reads without listening               | |
|   +-------------------------------------------------------+ |
|                                                              |
+--------------------------------------------------------------+
```

12 Security Measures Implemented

12.1 Security Features

- **Password Hashing**: bcrypt algorithm (salt + hash)
- **JWT Authentication**: Signed tokens with expiration
- **Token Refresh**: Secure token rotation
- **SQL Injection Prevention**: Parameterized queries (QueryBuilder)
- **CORS Configuration**: Controlled cross-origin requests
- **Rate Limiting**: Prevent brute force attacks

- **Input Validation**: All user inputs validated
- **HTTPS Ready**: Production SSL/TLS support
- **Environment Variables**: Secrets not in code
- **Email Verification**: OTP before account activation
- **Password Reset Flow**: Secure reset with expiry
- **Account Deactivation**: Soft delete option

12.2 Password Hashing Example

```python
def hash_password(self, password: str) -> str:
    salt = bcrypt.gensalt()
    return bcrypt.hashpw(password.encode('utf-8'), salt).decode('utf-8')

def verify_password(self, hashed: str, password: str) -> bool:
    return bcrypt.checkpw(password.encode('utf-8'), hashed.encode('utf-8'))
```

Listing 14: bcrypt Password Hashing

12.3 Parameterized Query Example

```python
# NEVER do this:
# cursor.execute(f"SELECT * FROM users WHERE email = '{email}'")

# ALWAYS do this:
cursor.execute("SELECT * FROM users WHERE email = %s", (email,))
```

Listing 15: SQL Injection Prevention

13 Deployment Configuration

13.1 PM2 Ecosystem Configuration

```python
module.exports = {
  apps: [{
    name: "career-navigator-api",
    script: "app.py",
    interpreter: "python3",
    instances: 1,
    autorestart: true,
    watch: false,
    max_memory_restart: "1G",
    env: {
      FLASK_ENV: "production",
    },
    error_file: "./logs/err.log",
    out_file: "./logs/out.log",
```

```
  }]
}
```

Listing 16: ecosystem.config.js

13.2 Nginx Configuration for WebSocket

```
server {
    listen 80;
    server_name api.yourdomain.com;

    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    location /socket.io/ {
        proxy_pass http://127.0.0.1:5000/socket.io/;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_buffering off;
        proxy_cache off;
    }
}
```

Listing 17: Nginx with WebSocket Support

14 What We Have Accomplished

14.1 Completed Features

Feature	**Status**
User Registration with Email Verification	Complete
JWT Authentication with Token Refresh	Complete
Role-Based Access Control (Job Seeker/Mentor/Admin)	Complete
Profile Management (Basic Info)	Complete
Education History CRUD	Complete
Work Experience CRUD	Complete
Profile Picture Upload	Complete
Job Listings with Filters	Complete

Job Search	Complete
Apply for Jobs with Cover Letter	Complete
View My Applications	Complete
Mentor Search	Complete
Send Mentorship Requests	Complete
Real-Time Chat with WebSocket	Complete
Typing Indicators	Complete
Online/Offline Status	Complete
Video Calling with Agora	Complete
Video Session History	Complete
In-App Notifications	Complete
Mark Notifications as Read	Complete
Search Users (Mentors/Seekers)	Complete
Submit Feedback with Rating	Complete
Dark/Light Theme Toggle	Complete
Change Password	Complete
Forgot Password (Email Reset)	Complete
Delete Account	Complete
Help & FAQ Section	Complete
Privacy Policy Page	Complete
About Us Page	Complete
Admin Dashboard (Partial)	In Progress
Advanced Analytics	Planned
AI Job Recommendations	Planned

Table 8: Feature Completion Status

14.2 Technical Achievements

- Complete full-stack application with **100+ Dart files** (frontend) and **50+ Python files** (backend)
- **50+ API endpoints** fully documented and tested
- **18 database tables** with proper relationships and indexes
- **Real-time WebSocket communication** with automatic REST fallback
- **JWT token refresh mechanism** with request queueing
- **Secure authentication** with bcrypt password hashing
- **Responsive UI** supporting both dark and light themes
- **Production-ready deployment** configuration with PM2 and Nginx

15 Lessons Learned

15.1 Technical Lessons

- **WebSocket vs REST**: WebSocket is superior for real-time features but requires more complex infrastructure; having REST fallback ensures reliability
- **Token Management**: Implementing proper token refresh with request queueing prevents user disruption and maintains security
- **State Management**: Provider pattern is simple and effective for medium-sized Flutter applications
- **Database Design**: Proper indexing and foreign key constraints significantly improve query performance
- **Error Handling**: Comprehensive error handling at all layers (frontend, backend, database) improves user experience

15.2 Challenges Overcome

- **Timer management for typing indicators**: Implemented proper cleanup to prevent memory leaks
- **WebSocket reconnection logic**: Handled network interruptions gracefully
- **Token refresh race conditions**: Implemented request queuing to handle concurrent requests during refresh
- **CORS configuration**: Configured properly for development and production
- **File uploads**: Implemented both Cloudinary (production) and local (development) fallback

16 Future Improvements

16.1 Planned Enhancements

1. **AI-Powered Job Recommendations**: Use machine learning to suggest relevant jobs
2. **Skill Assessments**: Add quizzes and skill tests for job seekers
3. **Resume Builder**: Built-in resume/CV creation tool
4. **Interview Preparation**: Mock interview feature with AI feedback
5. **Video Recording**: Record and store mentoring sessions
6. **Push Notifications**: Mobile push notifications using Firebase
7. **Social Login**: Google, LinkedIn, GitHub OAuth integration

8. **Admin Dashboard**: Complete user and content management
9. **Analytics Dashboard**: Track user engagement and platform metrics
10. **Schedule Video Calls**: Pre-scheduled video sessions with calendar integration

17 Conclusion

The Career Navigator platform successfully delivers a comprehensive career mentorship solution connecting job seekers with experienced mentors. The application demonstrates:

- **Professional Software Engineering**: Following SOLID principles, design patterns, and clean architecture
- **Full-Stack Proficiency**: Flutter frontend with Flask backend and MySQL database
- **Real-Time Features**: WebSocket chat with typing indicators and online status
- **Secure Authentication**: JWT tokens with refresh mechanism and bcrypt password hashing
- **Video Integration**: Agora SDK for one-on-one mentoring sessions
- **Production Ready**: PM2 process management and Nginx configuration

The project serves as an excellent demonstration of modern software development practices, from initial requirements gathering to final deployment.

Thank You!

For questions or inquiries, please contact the development team.